1. Хөгжүүлэлтийн орчныг бэлтгэх

React JSX кодыг бичиж ажиллуулахын тулд **Node.js** болон **npm (Node Package Manager)** суулгасан байх шаардлагатай.

Алхам 1: Node.js суулгах

Node.js албан ёсны вэбсайтаас татаж авч, суулгана.

Суулгасны дараа **терминал** дээр доорх командыг ажиллуулж шалгана:

```
node -v
npm -v
```

Хэрэв Node.js болон npm суусан бол тэдгээрийн хувилбар гарах болно.

Алхам 2: React төслийг үүсгэх

React төслийг `create-react-app` ашиглан үүсгэнэ.

```
npx create-react-app jsx-example
```

Энэ команд нь `jsx-example` нэртэй React төслийг үүсгэнэ.

Алхам 3: Төслийн фолдер руу орох

```
cd jsx-example
```

Алхам 4: Төслийг ажиллуулах

```
npm start
```

Энэ команд нь **React хөгжүүлэлтийн сервер** асааж, хөтөч дээр `http://localhost:3000` хаягаар нээнэ.

---

2. JSX ашиглан энгийн React компонент бичих

`src` фолдер дотор **`App.js` файлыг** нээнэ. Дараах JSX кодыг бичиж, хадгална.

```
import React from "react";

const App = () => {
  const message = "Hello, World!";
```

```
  return (
    <div>
      <h1>{message}</h1>
    </div>
  );
};

export default App;
```

---

3. Үндсэн `index.js`-д компонентыг дуудах

`src/index.js` файлыг нээнэ. Дараах байдлаар `App` компонентыг оруулж өгнө.

```
import React from "react";
import ReactDOM from "react-dom";
import App from "./App";

ReactDOM.render(<App />, document.getElementById("root"));
```

---

4. Програмыг ажиллуулах

Дахин **`npm start`** командыг терминал дээр ажиллуулна.

Гаралт:

```
Hello, World!
```

Энэ нь хөтөч дээр харагдах болно.

---

5. JSX даалгавар

Даалгавар 1: Хувийн мэдээлэл

Шаардлага:

- `UserProfile` нэртэй компонент үүсгэх.
- `name` болон `hobby` хувьсагчийг JSX дотор `{}` ашиглан харуулах.

```
const UserProfile = () => {
  const name = "John Doe";
  const hobby = "playing chess";

  return (
    <div>
      <h1>Hello, my name is {name}.</h1>
      <p>My favorite hobby is {hobby}.</p>
    </div>
  );
```

```
};

export default UserProfile;
```

`App.js`-д дараах байдлаар ашиглана.

```
import React from "react";
import UserProfile from "./UserProfile";

const App = () => {
  return (
    <div>
      <UserProfile />
    </div>
  );
};

export default App;
```

Даалгавар 2: Динамик жагсаалт

Шаардлага:

- `FavoriteFoods` нэртэй компонент үүсгэх.
- `foods` массив үүсгэж, `map()` ашиглан JSX дотор динамик жагсаалт үүсгэх.

```
const FavoriteFoods = () => {
  const foods = ["Pizza", "Sushi", "Burger"];

  return (
    <div>
      <h1>My favorite foods:</h1>
      <ul>
        {foods.map((food, index) => (
          <li key={index}>{food}</li>
        ))}
      </ul>
    </div>
  );
};

export default FavoriteFoods;
```

`App.js`-д нэмэх:

```
import React from "react";
import FavoriteFoods from "./FavoriteFoods";

const App = () => {
  return (
    <div>
      <FavoriteFoods />
    </div>
  );
};
```

```
export default App;
```

React Component гэж юу вэ?

Component гэдэг нь React-д **UI-ийн (хэрэглэгчийн интерфэйсийн) жижиг, дахин ашиглагдах хэсэг** юм.

React апп нь олон жижиг component-уудаас бүрддэг. Компонент бүр тусдаа функцтэй, харагдах байдалтай байж болно.

Component-ын онцлог

Шинж чанар	Тайлбар
Дахин ашиглагддаг	Нэг компонент олон газар ашиглаж болно
Бие даасан	Өөрийн төлөв (state) болон оролт (props)-той
Дүрслэлтэй	JSX ашиглан UI-г тодорхойлдог
Функц эсвэл Класс	function component, class component 2 төрөлтэй

---

2 төрлийн компонент байдаг

Төрөл	Тайлбар
Функц компонент	Энгийн JavaScript функц (→ useState, useEffect ашиглана)
Класс компонент	ES6 class ашиглан бичдэг (extends React.Component)

Функц компонент жишээ

jsx
CopyEdit
```
import React from "react";

const Welcome = () => {
  return <h1>Сайн байна уу!</h1>;
};

export default Welcome;
```

Ашиглах:

jsx
CopyEdit
```
<Welcome />
```

Class компонент жишээ

jsx
CopyEdit
```
import React, { Component } from "react";

class Welcome extends Component {
  render() {
    return <h1>Сайн байна уу!</h1>;
  }
}

export default Welcome;
```

 Ашиглах:

jsx
CopyEdit
```
<Welcome />
```

Компонентын бүтэц

jsx
CopyEdit
```
function Hello(props) {
  return (
    <div>
      <h2>Hi, {props.name}</h2>  {/* props ашиглаж байна */}
    </div>
  );
}
```

 Ашиглах:

jsx
CopyEdit
```
<Hello name="Naraa" />
```

Жишээ: Оюутны мэдээллийн компонент

jsx
CopyEdit
```
function StudentCard(props) {
  return (
    <div style={{ border: "1px solid gray", padding: "10px", margin: "5px" }}>
      <h3>Нэр: {props.name}</h3>
      <p>Анги: {props.grade}</p>
    </div>
  );
}
```

 Ашиглах:

jsx
CopyEdit
<StudentCard name="Бат" grade="10-р анги" />
<StudentCard name="Сараа" grade="11-р анги" />

 Компонентуудыг яагаад ашигладаг вэ?

✅ Кодыг **хялбар хувааж**, зохион байгуулах
✅ Дахин ашиглах боломжтой — "нэг бичээд олон ашиглах"
✅ Тестлэх, хөгжүүлэхэд хялбар
✅ Том төслүүдийг **бяцхан хэсгүүд** болгон хуваах боломжтой

State гэж юу вэ?

State гэдэг нь **компонентын дотоод төлөв** буюу тухайн агшинд хадгалагдаж буй **өгөгдөл, мэдээлэл** юм.

🔁 State өөрчлөгдөх үед React **автоматаар DOM-ийг дахин зурдаг (re-render)**.

🆚 **Props vs State**

Шинж	Props	State
Хаанаас ирдэг вэ?	Parent component-аас	Component дотор
Өөрчлөгдөх үү?	Үгүй (read-only)	Тийм (setState ашиглана)
Зорилго	Өгөгдөл дамжуулах	Дотоод өгөгдлийг хадгалах, удирдах

◆ **State ашиглах шалтгаан**

- Хэрэглэгчийн **интеракц** (товч дарах, текст оруулах) бүрийн хариуд UI-г шинэчлэх
- Тоолуур, форм, жагсаалт гэх мэт **динамик өгөгдөлтэй** UI хийх
- Тухайн component-ын **амьдралын мөчлөг** дэх төлөвийг хадгалах

✅ **Функциональ компонентийн жишээ (useState ашиглан)**

jsx
CopyEdit
import React, { useState } from "react";

```
const Counter = () => {
  const [count, setCount] = useState(0); // count = state, setCount = шинэчлэх функц

  const increase = () => {
    setCount(count + 1); // state-г шинэчилнэ
  };

  return (
    <div>
      <h2>Тоолуур: {count}</h2>
      <button onClick={increase}>Нэмэх</button>
    </div>
  );
};

export default Counter;
```

 Гаралт:

```
less
CopyEdit
Тоолуур: 0
[Нэмэх] —> дарснаар тоо нэмэгдэнэ
```

✅ Class Component дахь State-ийн жишээ

```
jsx
CopyEdit
import React, { Component } from "react";

class Counter extends Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0, // анхны төлөв
    };
  }

  increase = () => {
    this.setState({ count: this.state.count + 1 }); // state шинэчлэх
  };

  render() {
    return (
      <div>
        <h2>Тоолуур: {this.state.count}</h2>
        <button onClick={this.increase}>Нэмэх</button>
      </div>
    );
```

```
  }
}

export default Counter;
```

🔄 State өөрчлөгдөх үед юу болох вэ?

1. setState() эсвэл useState ашиглан state-г өөрчилнө.
2. React энэ өөрчлөлтийг мэдэрч, DOM-ийг **дахин зурдаг (re-render)**.
3. Хэрэглэгч шинэ мэдээллийг UI дээр харах боломжтой болдог.

🧪 Жишээ дасгал:

Даалгавар: Like товч бүхий компонент

- LikeButton нэртэй компонент үүсгэ
- likes нэртэй state ашиглаж тоо хадгал
- "Like" товч дарснаар likes 1-р нэмэгдэнэ

```
jsx
CopyEdit
import React, { useState } from "react";

const LikeButton = () => {
  const [likes, setLikes] = useState(0);

  return (
    <div>
      <p>Like: {likes}</p>
      <button onClick={() => setLikes(likes + 1)}>Like ❤️</button>
    </div>
  );
};

export default LikeButton;
```

🧠 State ашиглах түгээмэл тохиолдлууд

- Тоолуур (counter)
- Форм бөглөлт (input)
- Modal toggle (нээх/хаах)
- Dynamic жагсаалт (array state)
- Theme солих (dark/light mode)

Props гэж юу вэ?

Props гэдэг нь **"properties"** буюу **шинж чанар** гэсэн үгийн товчлол юм.
React-д **компонентод гаднаас өгөгдөл дамжуулах** зориулалттай механизм юм.

Props нь нэг чиглэлтэй өгөгдлийн урсгалыг (parent ➞ child) ашигладаг.

Онцлогууд

Тайлбар	Жишээ
Props нь **read-only** буюу зөвхөн унших зориулалттай	Компонент дотор props-ийг өөрчилж болохгүй
Props-ийг **parent (эцэг) компонент** дамжуулдаг	<ChildComponent name="Tom" /> гэх мэт
Props-ийг **child (хүүхэд) компонент** хүлээж авдаг	props.name гэж уншдаг

Жишээ 1: Props ашиглах энгийн жишээ

jsx
CopyEdit
```
// Parent компонент
import React from "react";
import Greeting from "./Greeting";

const App = () => {
  return <Greeting name="Naraa" />;
};

export default App;
```
jsx
CopyEdit
```
// Greeting компонент
import React from "react";

const Greeting = (props) => {
  return <h1>Сайн байна уу, {props.name}!</h1>;
};

export default Greeting;
```

Гаралт:

CopyEdit
```
Сайн байна уу, Naraa!
```

Class Component дотор props ашиглах

jsx
CopyEdit
```
import React, { Component } from "react";

class Greeting extends Component {
  render() {
    return <h1>Сайн байна уу, {this.props.name}!</h1>;
  }
}

export default Greeting;
```

Props ашиглах шалтгаан

1. **Өгөгдөл дахин ашиглах**: Компонент бүр өөр өгөгдөл авч өөрөөр харагдах боломжтой.
2. **Компонент хооронд өгөгдөл дамжуулах**: Эцэг компонентээс хүүхдэд өгөгдөл илгээж болно.
3. **Dynamic UI**: Хэрэглэгчийн нэр, нас, жагсаалт зэргийг props ашиглан харуулах.

Жишээ 2: Props ашиглан динамик жагсаалт

jsx
CopyEdit
```
const FruitList = (props) => {
  return (
    <ul>
      {props.fruits.map((fruit, index) => (
        <li key={index}>{fruit}</li>
      ))}
    </ul>
  );
};

// Ашиглах
<FruitList fruits={["Алим", "Банана", "Гүзээлзгэнэ"]} />
```

Гаралт:

diff
CopyEdit
```
- Алим
- Банана
```

- Гүзээлзгэнэ

Props vs State

Харьцуулалт	Props	State
Хаанаас ирдэг вэ?	Гадаад (parent component)	Дотоод (component дотор)
Өөрчлөгдөх боломжтой юу?	Үгүй (read-only)	Тийм (setState ашиглана)
Зорилго	Өгөгдөл дамжуулах	Component доторх динамик өгөгдлийг удирдах

.